Physics of Deep Learning

Part I: Bayesian learning and phase transitions

Tudor Dimofte

Iliad Intensive (Sept. 2026)

I. Bayesian learning and phase transitions
II. Networks at large width
III. Scaling laws
IV. Closing reading: learning mechanics

Questions about learning

How and when are **features** learned during training?

- How to quantify? How to even *define* features?
- How does this depend on architecture, # parameters, training data, ...
- How can we *steer* learning, emergent capabilities, etc.?

Physics-inspired approaches help identify **approximations** in which these questions can be studied, in a simple(r), quantitative, often analytic way.

The approximations typically involve **scaling** of some sort: sending various (hyper)parameters $\to \infty$.

Structure of the day

This lecture: Bayesian learning and statistical mechanics. A common approximation scheme, with a particularly strong connection to physics. I'll mainly talk about physics, and ask you to apply the same ideas to learning in the exercises!

Part II: Behavior of neural networks at large values of hyperparameters (especially: large width).

Part III: Neural scaling laws – one of the simplest robust empirical observations that any theoretical model needs to be able to explain.

Part IV: (if there's time) Reading & discussing a recent survey on the science of deep learning.

Statistical mechanics & statistical learning

We initialize networks randomly:

$$\theta \sim \pi \qquad (\theta = \text{parameters},\ \#\theta = N)\,.$$

Then we train with some version of gradient descent, roughly

$$\Delta\theta \approx -\eta\, \nabla_\theta L(\theta)\,.$$

A common approximation to GD is **Bayesian learning**

$$p_\beta(\theta) \ \propto\ e^{-\beta L(\theta)}\, \pi(\theta)\,.$$

- Interpret $e^{-\beta L(\theta)}$ as the *likelihood* of the training data, given the model at fixed θ.
- $\beta \approx$ how much training has been done.

An interpretation of β

For n samples $(x_i, y_i)_{i=1}^n$:

$$L_n(\theta) = \sum_{i=1}^{n} \ell_i(\theta)\,, \qquad \ell_i(\theta) = \ell\big(f_\theta(x_i),\, y_i\big)\,,$$

$$L_n(\theta) = n \cdot \underbrace{\frac{1}{n}\sum_{i} \ell_i(\theta)}_{\text{finite as } n\to\infty} \;\approx\; n\,\bar{L}(\theta)\,, \qquad \bar{L}(\theta) = \mathbb{E}_{(x,y)\sim q}\,\ell\big(f_\theta(x), y\big)\,.$$

For large n:

$$p_\beta(\theta) \;\propto\; e^{-\,n\beta\,\bar{L}(\theta)}\,\pi(\theta)\,.$$

- The effective parameter is $\beta' = n\beta$: *more data = more training = larger β' = "colder."*
- We'll often write $e^{-\beta\bar{L}}\pi$; but remember $\beta \to \infty$ as $n \to \infty$.

The analogy with thermal physics

Bayesian learning looks a lot like a **thermal** system, i.e. like near-equilibrium statistical mechanics:

$$\begin{aligned}
\theta \quad (\#\theta = N) &\leftrightarrow \text{microscopic state of a system} \\
&\quad\ (\text{velocities of } N \text{ gas particles; spins of } N \text{ electrons}) \\
L(\theta) &\leftrightarrow \text{energy } E(\theta) \quad (\text{or Euclidean action } S(\theta)) \\
p(\theta) \propto e^{-\beta L(\theta)} &\leftrightarrow \text{Boltzmann distribution: state of a system} \\
&\quad\ \text{that's been equilibrated with a bath at } T = 1/\beta
\end{aligned}$$

Note: to include the prior $\pi(\theta)$ in this setup, just redefine

$$\tilde{L} = L - \tfrac{1}{\beta} \log \pi \quad \text{s.t.} \quad e^{-\beta L(\theta)} \pi(\theta) = e^{-\beta \tilde{L}(\theta)}\,, \qquad \tilde{L}(\theta) \leftrightarrow E(\theta)\,.$$

The analogy opens up lots of useful **methodology**.

Terminology & common tricks

Partition function (of β & other hyperparameters):

$$Z(\beta) = \int d\theta\; e^{-\beta E(\theta)}\,, \qquad p_\beta(\theta) = \tfrac{1}{Z(\beta)}\, e^{-\beta E(\theta)}\,.$$

Suppose there are **observables** $\mathcal{O}_i(\theta)$ whose expectation values we want. Generalize with **sources** J:

$$Z(\beta, J) = \int d\theta\; e^{-\beta E(\theta) + \sum_i J^i \mathcal{O}_i(\theta)}\,.$$

Then

$$\mathbb{E}(\mathcal{O}_i) = \frac{\partial}{\partial J^i} \log Z\Big|_{J=0}\,, \quad \mathrm{Cov}(\mathcal{O}_i, \mathcal{O}_j) = \frac{\partial^2}{\partial J^i \partial J^j} \log Z\Big|_{J=0}\,,$$

and in general $\kappa^{(p)}(\mathcal{O}_{i_1}, \ldots, \mathcal{O}_{i_p}) = \partial_{J^{i_1}} \cdots \partial_{J^{i_p}} \log Z\big|_{J=0}$ is the p-th **cumulant**.

$F(\beta, J) := -\log Z(\beta, J)$ is the (sourced) **free energy**: the generating function of cumulants.

Cumulants of the energy itself

If the only observable we cared about was $E(\theta)$, we could just use $Z(\beta)$ (no sources), $F(\beta) = -\log Z(\beta)$:

$$\mathbb{E}(E) = \partial_\beta F\,, \qquad \mathrm{Var}(E) = -\partial_\beta^2 F\,, \qquad \kappa^{(3)}(E) = \partial_\beta^3 F\,, \ \ldots$$

We'll see these derivatives of F again in a little bit.

The analogy with physics also opens up a bag of well-developed **approximations**. We'll illustrate two in a classic physics example:

- Mean-field approximation, to help connect microscopic (individual param.) and macroscopic (large-scale) observables.
- Simplifications at large N
  ($N = \#$ parameters here, but other limits are similar)

We'll look at the *Ising model*; see Tong's *Statistical Field Theory* notes for more on the awesome physics of this example.

Physics example: the Ising model

Λ: a graph with N vertices $\{i\}$ and undirected edges $\{\langle ij \rangle\}$.

At each vertex, place a "spin" $\theta_i \in \{\pm 1\}$: a system with 2^N states, specified by $\theta_1, \ldots, \theta_N$.

$$E(\theta) \;=\; -J \sum_{\text{edges } \langle ij \rangle} \theta_i \, \theta_j \;-\; B \sum_{\text{vxs } i} \theta_i$$

- J: coupling strength (alignment with neighbors)
- B: external magnetic field (alignment with the field)

An interesting observable: the **magnetization**

$$m(\theta) = \frac{1}{N} \sum_{i=1}^{N} \theta_i \;\in\; [-1, 1] \qquad \text{(average spin in a given state)}\,.$$

The simplest *macroscopic* quantity describing the system.

Mean-field approximation

We'd like to **push forward** the distribution $p_\beta(\theta) \propto e^{-\beta E(\theta)}$ to a distribution $p_\beta(m)$ over the macroscopic m.

Key: the **mean-field approximation**. Suppose each spin θ_i only feels the *average* force of all other spins, not its particular neighbors — i.e. replace each spin's actual neighbor sum by d copies of the global average (d = average # of neighbors):

$$\sum_{\langle ij \rangle} \theta_i \theta_j \;=\; \frac{1}{2} \sum_i \theta_i \underbrace{\sum_{j \sim i} \theta_j}_{\approx\; d\, m} \;\approx\; \frac{d}{2}\, m \sum_i \theta_i$$

$$\Rightarrow \quad E(\theta) \;\approx\; -N\left(\frac{dJ}{2}\, m^2 + Bm\right).$$

Turns out to hold at large N and large $d \gtrsim 4$. Assuming this,

$$Z(\beta, J, B) \;=\; \sum_{\{\theta\}} e^{-\beta E(\theta)} \;\approx\; \int_{-1}^{1} dm\; e^{N\beta\left[\frac{dJ}{2} m^2 + Bm\right]} \underbrace{\Omega(m)}_{\#\{\theta:\; m(\theta) = m\}}$$

Entropy, and the reduction to one variable

Counting states at fixed m: $\Omega(m) = \binom{N}{\frac{1+m}{2}N}$, and Stirling gives

$$\log \Omega(m) \underset{N\to\infty}{\approx} N \underbrace{\left[\log 2 - \tfrac{1}{2}(1-m)\log(1-m) - \tfrac{1}{2}(1+m)\log(1+m)\right]}_{h(m)}$$

$h(m) = \frac{1}{N}\log(\#\text{ available states})$ is the **entropy** (per-site) at magnetization m: the effective measure after switching variables.

$$Z(\beta, J, B) = \int_{-1}^{1} dm \underbrace{e^{N\beta\left[\frac{dJ}{2}m^2 + Bm + \beta^{-1}h(m)\right]}}_{=:\ e^{-N\beta\, s(m)}}$$

$s(m)$: effective **action**, or free-energy density at m.

We've reduced the problem from N parameters to **one**.

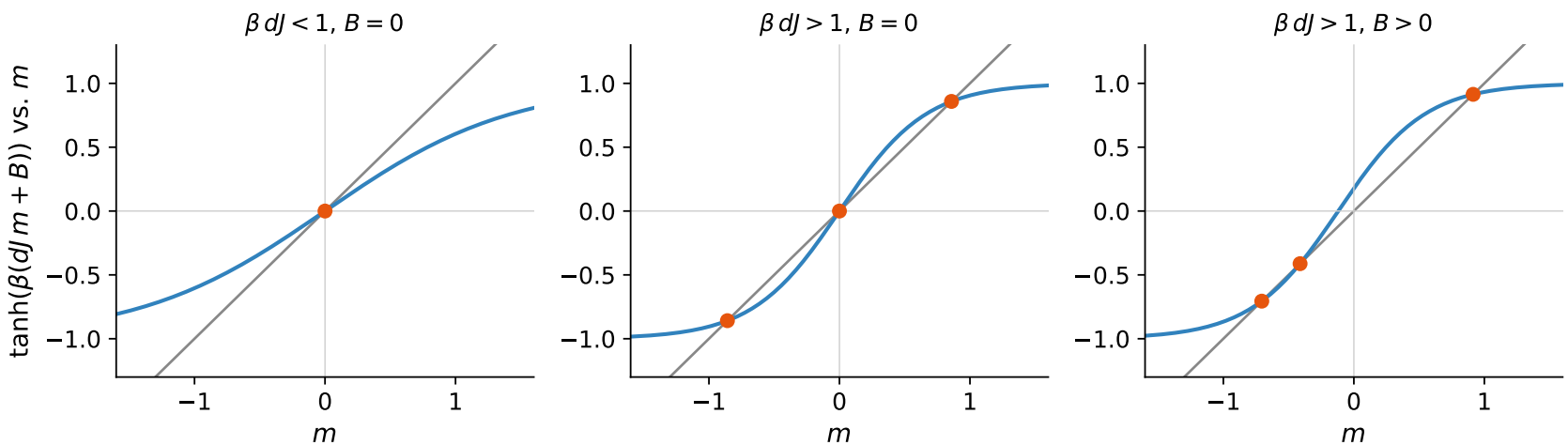
Saddle points

At large N, the integral concentrates around **minima** of s (saddle-point approximation):

$$\frac{\partial s}{\partial m} = 0 \quad \Longrightarrow \quad dJ\,\bar{m} + B = \beta^{-1}\tanh^{-1}(\bar{m})\,,$$

$$\text{i.e.} \qquad \boxed{\bar{m} = \tanh\left(\beta\left(dJ\,\bar{m} + B\right)\right)}$$

Solve graphically, for various β, J, B:



Self-consistency (an aside)

There's another path to the saddle-point equation. Look at the effective field/force felt by a single θ_i:

$$B^{\text{eff}} = -\partial_{\theta_i} E(\theta) = J \sum_{j \sim i} \theta_j + B \approx dJm + B \,.$$

The partition function for a single θ_i in this effective field becomes

$$Z_i = \sum_{\theta_i = \pm 1} e^{\beta B^{\text{eff}} \theta_i} = 2 \cosh \beta B^{\text{eff}} = 2 \cosh \beta (dJm + B) \,.$$

If the mean-field approx. is consistent, we'd better find

$$m \stackrel{!}{=} \mathbb{E}(\theta_i) = \tfrac{1}{\beta} \partial_B \log Z_i = \tanh \beta (dJm + B)$$

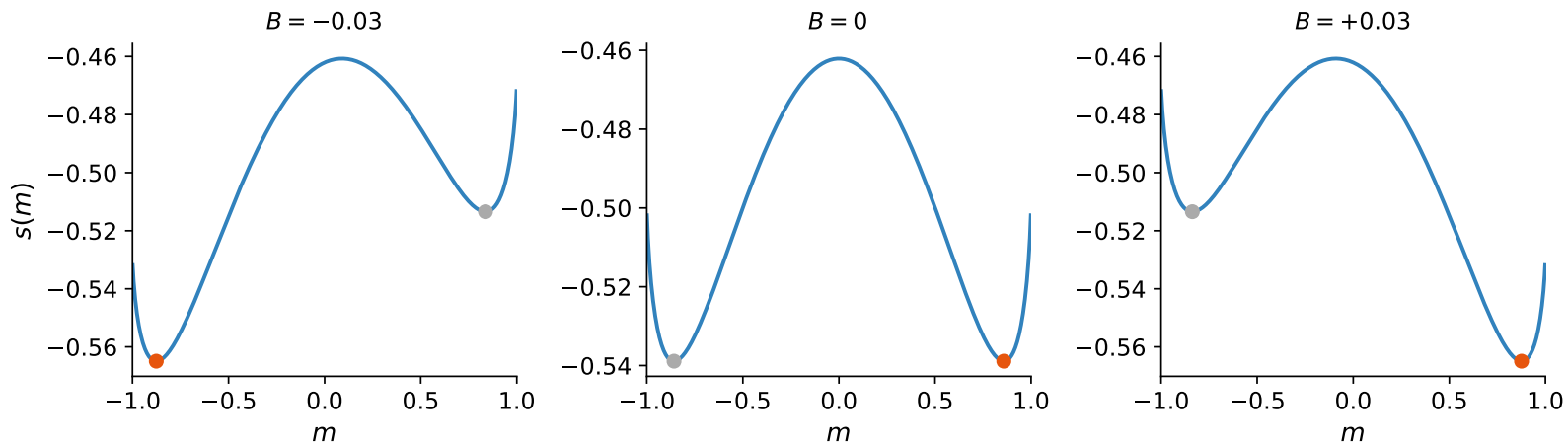
I.e. the average that comes *out* to equal the average we put in!

Phases & phase transitions

Analyze the structure of **minima of $s(m)$** — they control where the probability density accumulates. Rich structure as β, J, B are varied! Taylor-expanding $h(m) \approx \log 2 - \frac{m^2}{2} - \frac{m^4}{12}$:

$$s(m) \;\approx\; \text{const} - B\,m + \tfrac{1}{2}\left(\beta^{-1} - Jd\right) m^2 + \frac{\beta^{-1}}{12}\, m^4 + \cdots$$

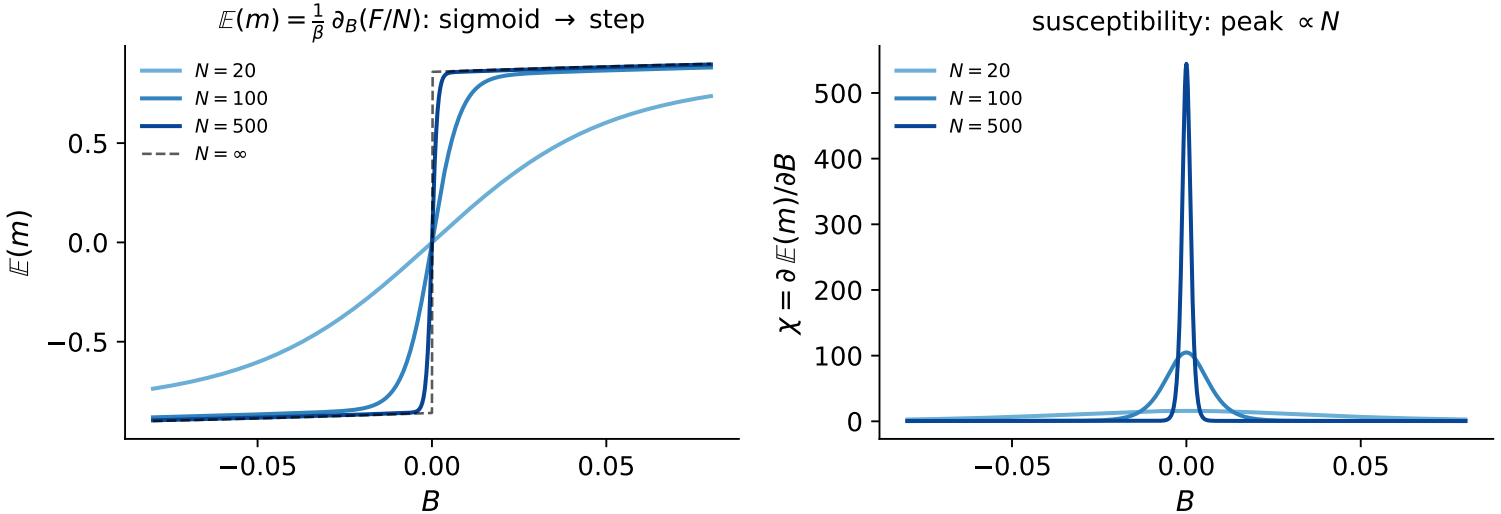
Take $Jd > \beta^{-1}$ (coupling beats temperature: the system tries to be *ordered*, neighboring spins align. But which way?)



They align **with the magnetic field B**: $\bar{m} \approx -1$ for $B < 0$, $\bar{m} \approx +1$ for $B > 0$. This is a **first-order phase transition** as B crosses $B_c = 0$.

Features of a first-order transition

The structure of this transition, as two isolated minima cross, is universal (independent of the model). Look at $\mathbb{E}(m) = \frac{1}{N\beta}\frac{\partial F}{\partial B}$:

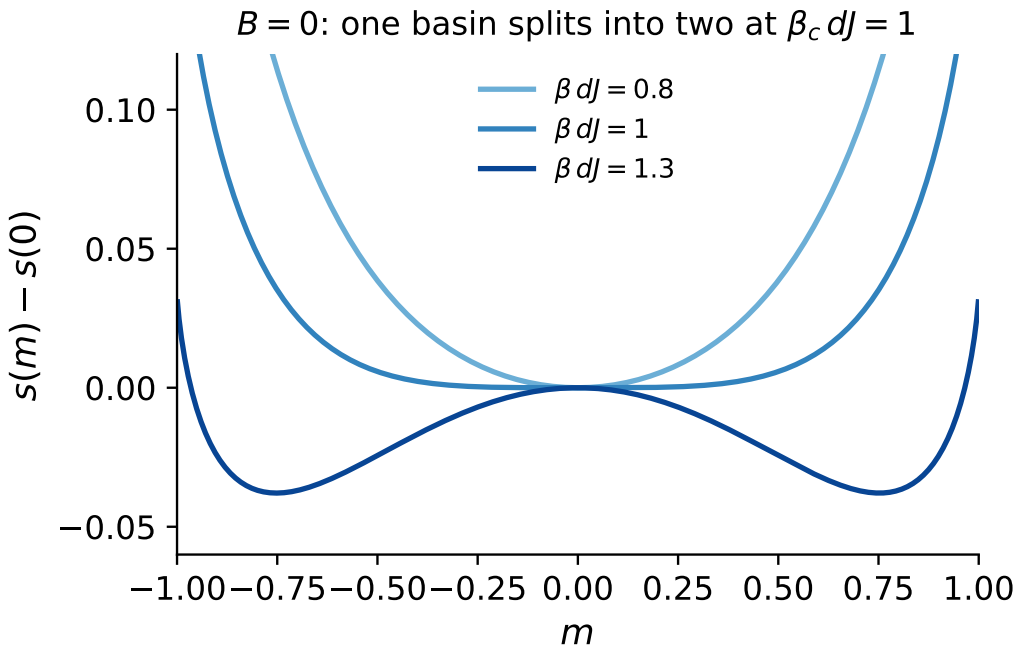


As $N \to \infty$, the saddle-point approx. strongly prefers m to sit in the lowest minimum, so $\mathbb{E}(m) = \beta^{-1}\partial_B(F/N)$ develops a discontinuity — the *definition* of a first-order phase transition. The "susceptibility" $\chi := \partial_B\mathbb{E}(m) = \beta^{-1}\partial_B^2(F/N)$ diverges to a δ.

A second-order transition, too

An *n-th order* phase transition = discontinuity in the n-th derivative of F/N, as $N \to \infty$.

In the Ising model this happens at fixed $B = 0$ (symmetric minima), as the temperature $T = \beta^{-1}$ crosses $T_c = Jd$:



One basin deforms and splits: the order parameter $\mathbb{E}(m)$ moves *continuously* but not smoothly ($\mathbb{E}(m) \propto (\beta - \beta_c)^{1/2}$).

Back to learning: Bayes vs. gradient descent

We don't train networks with Bayesian learning; we train them with **gradient descent**. Recall the expected analogy:

$$(*) \qquad \underbrace{n \to \infty}_{\text{samples / learning time}} \qquad \rightsquigarrow \quad \beta \to \infty \quad (T \to 0)\,.$$

- In Ising, this sweep would see the *second*-order transition.
- For *first*-order transitions, when we *dynamically* vary a parameter there is **hysteresis**: transitions are delayed. (One dynamical analogue in neural networks: the MSRJD formalism + DMFT — not in these lectures!)

The Langevin equation (an aside)

Another connection to gradient descent, worth knowing: the **Langevin equation**

$$\dot{\theta} = -\nabla_\theta L(\theta) + T^{1/2}\, d\xi \qquad \text{(Gaussian noise, variance } T\text{)}.$$

- Starting from any sufficiently generic prior, at late training time ($t \to \infty$) this converges to $p(\theta) \propto e^{-\frac{1}{T}L(\theta)}$: one can "Bayesian train" a network in practice.
  (More on this in Tong's *Kinetic Theory* lectures.)
- Alternatively, $T \to 0$ at finite time t produces gradient flow $\dot{\theta} = -\nabla_\theta L$.

  $\rightsquigarrow$ a *different* Bayesian $\leftrightarrow$ gradient connection.

Exercises (Part I)

See the handout. Do *one* of these.
Alternatively, go through formulas from the lecture and verify them.

0. Expectations and cumulants from the free energy.
1. **Anatomy of a first-order transition.** Work out the curves $\frac{\partial f}{\partial \beta}$ and $-\frac{\partial^2 f}{\partial \beta^2}$ $(f = F/N)$ for a generic first-order phase transition — and interpret them as the mean energy and the variance of the energy (heat capacity).
2. **The Ising perceptron.** Your turn! Analyze a simple learning problem and discover a first-order phase transition to perfect generalization.

References & further reading (Part I)

- D. Tong, *Statistical Field Theory* lecture notes — the physics behind this part (and *Kinetic Theory*, for the Langevin equation).
- A. Engel and C. Van den Broeck, *Statistical Mechanics of Learning* — the perceptron tradition; the Ising perceptron transition goes back to G. Györgyi (1990).
- The long lecture notes, *Physics of Deep Learning*, expand on everything today, with worked solutions.

Physics of Deep Learning
Part II: Scaling neural networks and feature learning

Tudor Dimofte

Iliad Intensive (Sept. 2026)

Main question

Given a neural network, analyzed either dynamically (SGD) or via equilibrium Bayesian methods: *what scaling limits simplify the analysis?* (And are these limits realistic?)

1. Lazy scaling at initialization
2. Detour/reminder: the NTK & features
3. Lazy scaling during training
4. Mean-field scaling (and μP)

Setup: the MLP

A network with parameters $\theta \in \mathbb{R}^P$. Keep it simple: an **MLP** (fully-connected network), input $x \in \mathbb{R}^d$, L hidden layers each of width N, one output $f(x)$.

Pre-activations in layer ℓ:

$$z_i^{(\ell)} = \sum_{j=1}^{N} w_{ij}^{(\ell)} \, \phi\big(z_j^{(\ell-1)}\big), \quad \text{except} \quad z_i^{(1)} = \sum_{j=1}^{d} w_{ij}^{(1)} \, x_j \;\; (\text{no } \phi),$$

$$f = z^{(L+1)} \qquad \text{so} \qquad \theta = \{w^{(1)}, \ldots, w^{(L+1)}\}.$$

(No biases — they don't add anything conceptual.)

Randomness at initialization

There is already randomness here, from random initialization: $\theta \sim \pi(\theta)$ (some *prior*).

Question: how does the randomness propagate to the values $f_\theta(x)$? Specifically: given the distribution π on θ's and fixed inputs $x^1, \ldots, x^k \in \mathbb{R}^d$, what is the **push-forward distribution** on

$$\big(f(x^1), \ldots, f(x^k)\big) \ ?$$

This simplifies greatly as $N \to \infty$.

But exactly *how* it simplifies depends strongly on π (as well as how learning rate, L, etc. scale with N).

Lazy initialization ("fan-in norm")

Lazy init: try to keep all pre-activations $z^{(\ell)} = O(1)$.

$$z_i^{(\ell)} = \sum_{j=1}^{N} \underbrace{w_{ij}^{(\ell)}}_{N \text{ random coeffts}} \underbrace{\phi\big(z_j^{(\ell-1)}\big)}_{O(1)}$$

$$\Longrightarrow \quad \text{take} \quad w_{ij}^{(\ell)} \sim \frac{1}{\sqrt{N}}\,, \qquad w_{ij}^{(1)} \sim \frac{1}{\sqrt{d}}\,.$$

Let's implement this explicitly: for $\ell > 1$ set

$$z_i^{(\ell)} = \frac{1}{\sqrt{N}} \sum_{j=1}^{N} w_{ij}^{(\ell)}\, \phi\big(z_j^{(\ell-1)}\big)\,, \qquad w_{ij}^{(\ell)} \sim \mathcal{N}(0,1)\,,$$

so all the weights are now of order one.

Gaussian processes

Definition. Given $\mu : \mathbb{R}^d \to \mathbb{R}$ and $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ (the **kernel**), a **Gaussian process** is a random function $f : \mathbb{R}^d \to \mathbb{R}$ such that for any $(x^1, \ldots, x^k)$, the joint distribution of $\big(f(x^1), \ldots, f(x^k)\big)$ is Gaussian with mean $\big(\mu(x^1), \ldots, \mu(x^k)\big)$ and covariance matrix

$$\begin{pmatrix} K(x^1, x^1) & \cdots & K(x^1, x^k) \\ \vdots & & \vdots \\ K(x^k, x^1) & \cdots & K(x^k, x^k) \end{pmatrix}.$$

Equivalently: mean and covariance exist, but *all higher cumulants vanish* (recall the cumulants of Part I — here the observables are the $f(x^i)$ themselves).

Theorem: lazy init ⇒ Gaussian process

Theorem (Neal; Lee et al.; Yang; Hanin...). Fix L and x's. With lazy init, as $N \to \infty$, f converges to a Gaussian process, with recursively defined kernel:

$$K^{(\ell+1)}(x, x') = \mathbb{E}_{z \sim K^{(\ell)}} \Big[\phi\big(z(x)\big)\, \phi\big(z(x')\big) \Big],$$

with base case $K^{(1)}(x, x') = \frac{1}{d} x \cdot x'$. *Sketch:* the first layer is a sum of Gaussians — exactly Gaussian, with kernel $K^{(1)}$. Induct on layers: *conditioned* on layer ℓ, the $z_i^{(\ell+1)}$ are i.i.d. Gaussian with the *random* kernel $\widehat{K}^{(\ell)} = \frac{1}{N} \sum_j \phi(z_j^{(\ell)}(x))\, \phi(z_j^{(\ell)}(x'))$. Key: as $N \to \infty$ its variance $\to 0$, so it becomes *deterministic* — and Gaussianity propagates.

The recursion has an analytic form for $\phi = \mathrm{ReLU}$ and erf.

(There is also a slick physics derivation by path integrals — left as an exercise for those who like them; see the long notes, §3.)

Proof sketch, I: the first layer is exactly Gaussian

Fix the inputs $x^1, \dots, x^k$. The first layer, with the $1/\sqrt{d}$ explicit, is

$$z_i^{(1)}(x) = \frac{1}{\sqrt{d}} \sum_{j=1}^{d} w_{ij}^{(1)} x_j \,, \qquad w_{ij}^{(1)} \sim \mathcal{N}(0,1) \;\; \text{i.i.d.}$$

For fixed i, this is a linear combination of independent Gaussians with *fixed* coefficients $x_j/\sqrt{d}$ — so $\left(z_i^{(1)}(x^1), \dots, z_i^{(1)}(x^k)\right)$ is **exactly jointly Gaussian** (no large-N limit needed), with mean zero and kernel

$$K^{(1)}(x,x') = \mathbb{E}\left[z_i^{(1)}(x)\, z_i^{(1)}(x')\right] = \frac{1}{d} \sum_{j,j'=1}^{d} x_j\, x'_{j'} \underbrace{\mathbb{E}\left[w_{ij}^{(1)} w_{ij'}^{(1)}\right]}_{\delta_{jj'}} = \frac{x \cdot x'}{d} \,.$$

Moreover, distinct neurons $i \neq i'$ use *disjoint* sets of weights, so the $z_i^{(1)}$ are i.i.d. across i: layer 1 is N independent copies of one Gaussian process.

Proof sketch, II: induction on layers

Inductive hypothesis: as $N \to \infty$, the $z_j^{(\ell-1)}$ become i.i.d. GPs with deterministic kernel $K^{(\ell-1)}$.
Conditioned on layer $\ell - 1$, the coefficients $\phi\big(z_j^{(\ell-1)}\big)$ are frozen, so $z_i^{(\ell)} = \frac{1}{\sqrt{N}} \sum_j w_{ij}^{(\ell)}\, \phi\big(z_j^{(\ell-1)}\big)$ is *again* linear in Gaussian weights: exactly Gaussian, i.i.d. across i, with kernel

$$\widehat{K}^{(\ell)}(x, x') = \frac{1}{N} \sum_{j=1}^{N} \phi\big(z_j^{(\ell-1)}(x)\big)\, \phi\big(z_j^{(\ell-1)}(x')\big)\,.$$

Un-conditionally, $\widehat{K}^{(\ell)}$ is a **random variable** — but an average of N i.i.d. $O(1)$ terms (by the inductive hypothesis), so $\mathrm{Var}\big[\widehat{K}^{(\ell)}\big] = O\big(\frac{1}{N}\big)$, with mean

$$\mathbb{E}\big[\widehat{K}^{(\ell)}\big] = \mathbb{E}_{z \sim K^{(\ell-1)}}\big[\phi\big(z(x)\big)\, \phi\big(z(x')\big)\big] = K^{(\ell)}(x, x')\,.$$

Key: as $N \to \infty$ the variance $\to 0$: the kernel becomes *deterministic*. A Gaussian mixture with deterministic kernel is Gaussian — and the neurons, coupled only through $\widehat{K}^{(\ell)}$, stay independent: Gaussianity propagates.

Lesson — and the L/N expansion

Lesson: the $N \to \infty$ limit with lazy init is *boring* (but tractable!) at init. It also turns out boring(ish) in training.

But: the $N \to \infty$ limit does *not* commute with $L \to \infty$. If L is large, each successive layer gets larger fluctuations in its kernel, and they don't die off.

Hanin, Roberts–Yaida: for $N, L \to \infty$ networks the relevant expansion parameter is $\boxed{L/N}$. We've just analyzed $L/N = 0$.

- *Small* (but nonzero) L/N: no longer Gaussian — cumulants $K^{(4)}(f(x^1), \ldots, f(x^4))$ nonzero, but *controllable*.
- *Large* L/N: each successive cumulant is even larger; uncontrollable outputs.

This translates to **training** (SGD, GD, grad flow, even Bayesian):

$\frac{L}{N} = 0$: network is linear (features fixed);
$\frac{L}{N}$ small: features can be learned;
$\frac{L}{N}$ large: chaotic, divergent flows $\to$ untrainable network.

Detour: gradient flow and the NTK

To analyze training and quantify "features", we'll introduce a geometric object known as the Neural Tangent Kernel (NTK). (Jacot, Gabriel, Hongler 2018)

Recall: on a manifold X with smooth $h : X \to \mathbb{R}$ and a non-degenerate metric $g = g_{ab}(x)\, dx^a dx^b$, **gradient flow** is

$$\frac{dx^a}{dt} = -\, g^{ab}(x)\, \frac{\partial h}{\partial x^b} \qquad \text{(the flow of the vector field } -\, g^{-1}(dh)\text{)}\,.$$

You *need* a metric to define gradient flow, descent, etc.

In the parameter space of neural networks, the metric is assumed to be **Euclidean**: $g_\theta = \delta_{ij}\, d\theta^i d\theta^j$. (First source of inductive bias!)
Training (as gradient flow) is $\dot{\theta}^i = -\, \delta^{ij} \partial_j L(\theta)$.

Push the flow to function space: the NTK

Suppose $\{x^\mu\}_{\mu=1}^n$ are fixed inputs, with network outputs $f^\mu = f_\theta(x^\mu)$ and "truth" values y^μ. Push forward the flow from $\theta \in \mathbb{R}^P$ to $\{f^\mu\} \in \mathbb{R}^n$: the standard coordinate transformation of the *inverse metric* gives

$$\delta^{ij} \text{ on } \mathbb{R}^P \quad \rightsquigarrow \quad \Theta^{\mu\nu} = \frac{\partial f^\mu}{\partial \theta^i}\, \delta^{ij}\, \frac{\partial f^\nu}{\partial \theta^j} = \nabla_\theta f^\mu \cdot \nabla_\theta f^\nu \text{ on } \mathbb{R}^n \,,$$

$$\dot\theta = -\nabla_\theta L \quad \rightsquigarrow \quad \dot f^\mu = -\underbrace{\Theta^{\mu\nu}}_{\textbf{NTK}}\ \underbrace{\partial_{f^\nu} L}_{\text{residual}} \,.$$

Compactly, $\Theta = JJ^\top$ with $J = \nabla_\theta f$ the Jacobian.
For MSE loss

$$L = \frac{1}{2}\sum_\mu |f^\mu - y^\mu|^2 \quad \Rightarrow \quad \partial_{f^\mu} L = f^\mu - y^\mu =: \Delta^\mu \,.$$

"The NTK controls the geometry of optimization."

Features, via the SVD

At a given θ, the Jacobian J says how the network values (on training data) respond to a shift in parameters. Take its SVD:

$$J = U \Lambda V^\top, \qquad \Lambda = \left(\operatorname{diag}(\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots) \mid 0 \right).$$

- columns of V: modes in *parameter* space;
- columns of U: modes in *function values* $\to$ **"features"**;
- $\Theta = JJ^\top = U \Lambda \Lambda^\top U^\top$: the features are the **eigenvectors of the NTK**, with eigenvalues λ_k.

This is a commonly used theoretical notion of *feature*.

Cf. Activation-space or SAE features, grokked capabilities...

An empirical comparison: see J. Lin, *Feature Identification via the Empirical NTK*, arXiv:2510.00468: NTK eigenvectors matching features actually learned in modular arithmetic and a small language model.

How big is the NTK, in lazy scaling?

Back to training at large N! What does the NTK look like?

Try $L = 1$: $\quad f(x) = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} c_i \, \phi(a_i \cdot x), \qquad a, c \sim \mathcal{N}(0, 1).$

$$\nabla f = \frac{1}{\sqrt{N}} \bigoplus_i \Big(\phi(a_i \cdot x) \,, \;\; c_i \, \phi'(a_i \cdot x) \, x \Big)$$

$$\Theta^{\mu\nu} = \frac{1}{N} \sum_{i=1}^{N} \Big[\phi(a_i \cdot x^\mu) \, \phi(a_i \cdot x^\nu) + c_i^2 \, \phi'(a_i \cdot x^\mu) \, \phi'(a_i \cdot x^\nu) \; x^\mu \cdot x^\nu \Big] = O(1) \,.$$

Similarly, at finite depth L and large N (with the $1/\sqrt{N}$'s explicit): Θ = sum of L terms, each $O(1)$ — so $\Theta = O(1)$, positive-definite.

Warning: in the $w \sim \mathcal{N}(0, \frac{1}{N})$ parametrization one finds $\Theta = O(N)$; the order-one flow there is $\dot{\theta} = -\frac{\eta}{N} \nabla L$. Rescaling parameters rescales the metric on parameter space — think of it as rescaling the *learning rate*. What matters: $O(1)$ changes in f^μ; figure out the size of Θ and scale η accordingly.

Theorem: lazy training is lazy

Theorem (Du et al., Jacot et al., . . .). Fix depth, data, and a sufficiently overparametrized lazy network, trained by gradient flow with the scalings above. Despite the fact that $f^\mu = O(1)$ moves and fits the data:

- The network **linearizes** around initialization:

$$f_\theta(x) = f_{\theta_0}(x) + (\theta - \theta_0) \cdot \nabla_\theta f(x)\big|_{\theta_0} + O\Big(\frac{1}{\sqrt{N}}\Big)\,;$$

- the NTK **freezes**: $\dfrac{\|\dot\Theta\|}{\|\Theta\|} = O\Big(\dfrac{1}{\sqrt{N}}\Big)\,;$

- $f(x^\mu) \to y^\mu$ *exponentially fast* in training time, with rates determined by the spectrum of Θ.

Why? (sketch)

1. $\|\nabla^2 f\|_{\mathrm{op}} = O\big(\frac{1}{\sqrt{N}}\big)$: a big matrix, but *sparse* — block-diagonal per neuron, each block carrying the explicit $\frac{1}{\sqrt{N}}$. (In fact $O(\frac{1}{N})$ after cancelling fluctuations, but $\frac{1}{\sqrt{N}}$ is enough.)
2. Taylor-expand: the quadratic term is $\|\Delta\theta\|^2 \, \|\nabla^2 f\|_{\mathrm{op}} \approx O(1) \times O\big(\frac{1}{\sqrt{N}}\big)$, using $\|\Delta\theta\| = O(1)$ from $\|\dot\theta\| = \|\sum_\mu \Delta^\mu \nabla f^\mu\| \sim \sqrt{\Theta} = O(1)$.
3. $\|\dot\Theta\| \le 2 \, \|\nabla^2 f\|_{\mathrm{op}} \, \|\dot\theta\| \, \|\nabla f\| \to 0$: the kernel freezes.
4. With Θ constant, the flow solves itself: $\dot\Delta^\mu = -\Theta^{\mu\nu}\Delta_\nu$, so in the NTK eigenbasis ($\Theta = U\Lambda\Lambda^\top U^\top$)

$$(U^\top \Delta)_k(t) = e^{-\lambda_k t} \, (U^\top \Delta)_k(0) \ :$$

the k-th mode/feature is learned in time $\boxed{t_k \approx 1/\lambda_k}$.

Lazy learning = kernel regression

This is called **lazy learning**: it uses the (*random*!) features from initialization to approximate the actual features in the data.
Overparametrized system, so that's fine — but it only *memorizes*.

[The learning algorithm here is simple and well known: *kernel regression*.]

This is **not** a good model for actual neural networks, which *change* their NTK (phase transitions!) to adapt to the training data and generalize intelligently.
Remarkably, it's enough to predict some basic behavior (Part III).

Theoretical setups that do better? Yes! Simplest options:

- small $L/N > 0$ [see e.g. Roberts–Yaida–Hanin];
- a *different init*: we'll look at **mean-field scaling**

  [Generalized to μP (maximal-update param.), Yang & Hu 2019]

Mean-field scaling

For an $L = 1$ network, change $\frac{1}{\sqrt{N}} \to \frac{1}{N}$:

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} c_i \, \phi(a_i \cdot x) \,, \qquad c_i, \, a_{ij} \sim \mathcal{N}(0, 1) \,,$$

and send $N \to \infty$. Main differences from the lazy regime:

- At init, $f(x) \to 0$: the Gaussian process *collapses*.
- In training, $\Theta^{\mu\nu} = O\left(\frac{1}{N}\right)$ — which *forces* the learning rate to scale as N for anything to happen:

$$\dot{\theta} = -N \, \nabla_\theta L \,, \qquad \dot{f}^\mu = -N \, \Theta^{\mu\nu} \Delta^\nu \,.$$

$\rightsquigarrow$ a nonlinear (in parameters) network, with an **adaptive NTK**.
Why does the NTK move now?

$$\frac{|\dot{\Theta}|}{|\Theta|} \lesssim \underbrace{N}_{\text{l.r.}} \cdot \|\nabla^2 f\|_{\text{op}} = N \cdot O\left(\frac{1}{\sqrt{N}} \cdot \frac{1}{\sqrt{N}}\right) = O(1) \,,$$

so it *can* move; a closer analysis of $\dot{a}_i, \dot{c}_i$ shows that it *does*.

Why the name "mean field"?

Think of $(a_i, c_i)_{i=1}^N$ as parameters for N **particles**, each contributing $g_i^\mu := c_i\,\phi(a_i \cdot x^\mu)$ to $f(x^\mu)$. The loss/energy is

$$L = \frac{1}{2}\sum_{\mu=1}^{n}\big(f(x^\mu) - y^\mu\big)^2 = \frac{1}{N}\sum_{\mu=1}^{n}\Big[\frac{1}{2N}\sum_{i,j=1}^{N} g_i^\mu\, g_j^\mu - y^\mu \sum_{i=1}^{N} g_i^\mu\Big] + \text{const}$$

Same form as the mean-field approximation of Ising! Weak all-to-all couplings, depending only on the **averages**

$$G^\mu = \frac{1}{N}\sum_i g_i^\mu \;: \qquad L = \frac{1}{\cancel{N}}\cdot N \sum_\mu \Big[\tfrac{1}{2}(G^\mu)^2 - y^\mu G^\mu\Big].$$

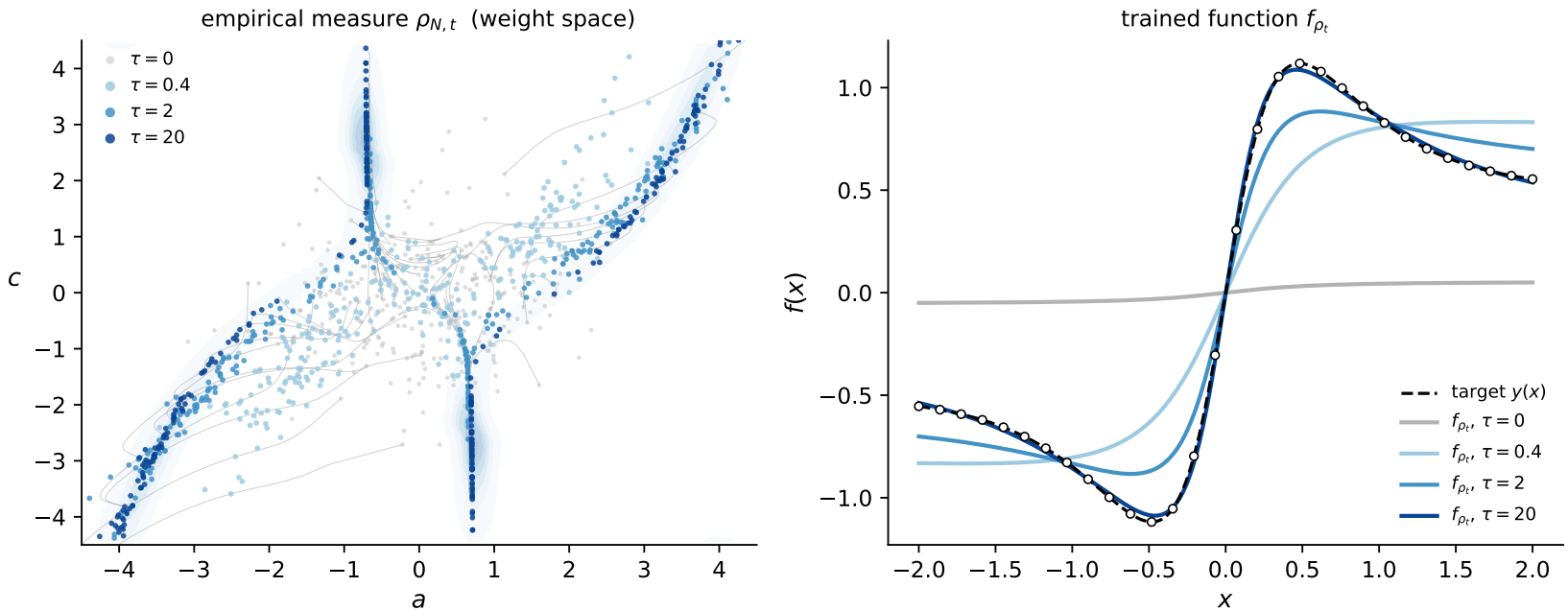
Theorem. As $N \to \infty$, at all training times t, the distribution of particles converges,

$$\frac{1}{N}\sum_i \delta(a - a_i)\,\delta(c - c_i) \;\longrightarrow\; \mu_t(a, c) \;:$$

a *smooth, deterministic* measure obeying a deterministic PDE.

(Details, and a Bayesian attack on the same system: long notes, §5.)

Feature learning, watched directly



Training in mean-field scaling transports the *cloud of neurons* (a_i, c_i) (left: dots colored by time, over the large-N density), which condenses onto the low-dimensional structures realizing the target — learning its features — while the network function converges (right). This is the model of Exercise 2.

Exercises (Part II)

See the handout:

0. **Depth-to-width ratios in the wild.** Look up L/N for open-weight LLM families: is everyone training in the "controlled correlations" window?
1. **Bayesian lazy learning.** The Bayesian-learning version of the lazy story: eigenmodes of the GP kernel at init are learned with characteristic "time" $\beta_k = 1/\lambda_k$ — the equilibrium twin of $t_k = 1/(\eta\lambda_k)$.
2. **Numerics: lazy vs. mean-field.** Train one small network in both scalings. Watch the NTK freeze in one and move in the other, and watch the particle cloud (a_i, c_i) condense onto features.

References & further reading (Part II)

- B. Hanin, *Mathematics of Deep Learning* lectures — large N `https://www.youtube.com/watch?v=4SVWo5kspR8`
- D. Roberts, S. Yaida, and B. Hanin, *The Principles of Deep Learning Theory* — the L/N expansion.
- Z. Ringel, N. Rubin, E. Mor, M. Helias, and I. Seroussi, *Applications of Statistical Field Theory in Deep Learning*.
- K. Spiliopoulos, R. Sowers, and J. Sirignano, *Mathematical Foundations of Deep Learning Models and Algorithms* — mean-field limits.
- The long lecture notes, *Physics of Deep Learning*, §3–§5, with worked solutions.

Physics of Deep Learning
Part III: Neural scaling laws

Tudor Dimofte

Iliad Intensive (Sept. 2026)

Organizing question

The test loss of large models falls as a clean power law in every resource spent on it. *Where do the exponents come from?*

The empirical laws (Kaplan et al. 2020)

For autoregressive language models, when not bottlenecked by the other resources, test loss obeys

$$L(N) \propto N^{-\alpha_N}\,, \qquad L(D) \propto D^{-\alpha_D}\,, \qquad L(C) \propto C^{-\alpha_C}\,,$$

with

- N = parameters (non-embedding)
- D = dataset size (in tokens)
- C = training compute (in FLOPs).

(Note: for single-epoch training, $C \approx 6ND$, with N & D optimally allocated.)

Measured exponents, holding over many orders of magnitude:

$$\alpha_N \approx 0.076\,, \qquad \alpha_D \approx 0.095\,, \qquad \alpha_C \approx 0.050\,.$$

Universal form; data-dependent exponents

The same power-law *form* holds across modalities (Henighan et al. 2020) — but the *exponents* follow the data:

modality	α_N	α_C
language	0.070	0.048
images 32×32	0.13	0.10
images 8×8	0.24	0.19
video	0.24	0.14
math (procedural)	0.16	0.17

- **Data matters**: exponents vary by a factor of $\sim$3–5 across modalities.
- **Architecture barely matters**: within reasonable architectures, changing shape (depth vs. width) or family moves constants and plateaus, not exponents. Whatever sets the exponent lives in the *data*.

Joint laws and bottlenecks

Single-resource laws assume the *other* resources are unlimited. Joint fits describe the crossover to being **bottlenecked**: loss is governed by whichever resource is scarcest. Two proposals:

Kaplan et al.: $L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}$

Chinchilla (Hoffmann et al. 2022): $L(N, D) = E + \frac{N_c}{N^{\alpha_N}} + \frac{D_c}{D^{\alpha_D}}$

- E: data's intrinsic noise floor, irreducible entropy (≈ 1.69 for text)
- Both forms reduce to single-resource power laws as either N or $D \to \infty$, removing bottleneck.
- The difference matters for compute-optimal scaling: at fixed $C \approx 6ND$, how to balance N and D?

Compute-optimal scaling: Kaplan vs. Chinchilla

Kaplan et al. 2020	$N \propto C^{0.73}$, $D \propto C^{0.27}$: *grow the model*
Chinchilla 2022	$N \propto C^{0.5}$, $D \propto C^{0.5}$: *grow both equally*
	(rule of thumb: ~ 20 tokens per parameter)

Which is right? Chinchilla's is now the accepted baseline. The discrepancy was analyzed & understood by Porian et al. 2024.

Note: In practice, frontier models now *over*-train relative to Chinchilla (more data per parameter), because smaller models are cheaper at *inference* time.

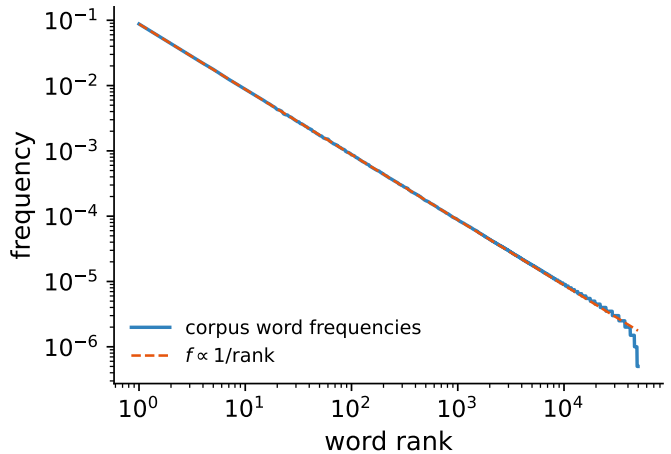
Name: *Chinchilla* was DeepMind's 70B model — trained with 4× the data of its 280B predecessor *Gopher* at the same compute, and beating it.

Where do the laws come from?

Basic answer: the loss curve is reading off the **coarse distribution of structure in the data**: power laws in, power laws out.

Simplest example: **Zipf's law**. Rank the words of a corpus by frequency, $f(\text{rank } k) \propto 1/k$.

(Similar: fact mentions, n-grams, image features . . . $\propto k^{-\alpha}$)



Theoretical status: many models derive the *single-resource* laws from data structure (now fairly clear); a few derive *multi-resource* laws. Still missing: the data's exponents from first principles; joint laws with full feature-learning dynamics; extending to emergent *capabilities*. Also missing theory for empirical RL laws (beyond this lecture).

A survey of theoretical proposals

proposal	data model / "features"	NN framework	exponents?	multi-param?
Sharma–Kaplan 2020	smooth target on d-dim. manifold	piecewise-linear interpolation	yes: $\alpha_N \approx 4/d$	no (N only)
kernel eigenlearning; Bahri et al. 2021; Bordelon et al.; Sollich (GP)	power-law kernel spectra + target coefficients	frozen kernel: NTK / GP-Bayesian	in terms of spectral exponents	partially: resolution vs. variance
Maloney–Roberts–Sully 2022	latent power-law covariance, extended by features	random feature model (frozen NTK) at $t \to \infty$	in terms of data spectral exp.	yes: joint $L(N, D)$; duality $N \leftrightarrow D$
quantization; Michaud et al. 2023	discrete skill "quanta," Zipf-used, learned in order	none (counting argument)	in terms of Zipf α; relation $\frac{1}{\alpha_D} = \frac{1}{\alpha_N} + 1$	separate N, D, C laws
percolation; Brill 2024	subtask clusters at criticality (power-law sizes)	general learner + regression estimates	from percolation critical exponents	two regimes; not joint
DMFT; Bordelon–Atanasov–Pehlevan 2024	power-law (source/capacity) spectra	DMFT of gradient-flow training, w/ frozen NTK	yes, incl. time exponent	fully joint $L(t, N, D)$ + compute frontier

Reading the table

- **Everyone agrees on the origin**: exponents are inherited from a power-law tail *in the data*. The models differ in what the tail is a tail *of*, and in how much network machinery is retained.
- Bayesian/equilibrium routes (GP learning curves; Ringel et al.'s effective-kernel methods at finite width) and dynamical routes (DMFT) give matching exponents where they overlap. *Cf.* Part I "two limits of one system".
- Only the static (MRS) and dynamical (DMFT) field-theory models currently predict *joint* laws — and DMFT derives a Chinchilla-like compute-optimal frontier. But both of these work in a frozen-NTK regime.
- None of them computes the *data's* exponent from first principles. Theory converts data statistics into loss curves; the statistics themselves are an empirical input.

The quantization model (Michaud et al. 2023)

A minimal model: no architecture, no optimizer, one tail sum.
Three assumptions:

- **QH1 (discreteness).** What a network learns decomposes into enumerable **quanta** — basic skills or facts ("carry the one"). Each is fully learned or not at all.
- **QH2 (ordered acquisition).** Models learn quanta in order of decreasing usefulness: a given model knows the first m.
- **QH3 (Zipfian use).** Quantum k is *used* on a fraction $p_k \propto k^{-(\alpha+1)}$ of data samples, and knowing it reduces the loss there by a constant Δ.

So a model that has learned m quanta has expected loss

$$L_m = L_\infty + \Delta \sum_{k>m} p_k \,.$$

The analysis boils down to asking **how each resource limits** m.

One law, worked; two laws, yours

$$\textit{Tail sum:}\quad L_m - L_\infty = \Delta \sum_{k>m} p_k \;\propto\; \int_m^\infty k^{-(\alpha+1)}\,dk \;\propto\; m^{-\alpha}\,.$$

Parameter-limited: if each quantum costs a constant capacity of c parameters, a size-N model can hold $m = N/c$ quanta:

$$L(N) - L_\infty \;\propto\; N^{-\alpha} \qquad \Longrightarrow \qquad \boxed{\alpha_N = \alpha}$$

— the model-size exponent *is* the data's Zipf exponent.

Exercise

Derive the other two laws yourself — the *data-limited* and *training-limited* exponents — and test the model against Kaplan's measured values. Full walkthrough on the Part III exercise sheet.

References & further reading (Part III)

- J. Kaplan et al., *Scaling Laws for Neural Language Models* (2020); T. Henighan et al. (2020) for other modalities.
- J. Hoffmann et al., *Training Compute-Optimal Large Language Models* (2022) — Chinchilla; T. Porian et al. (2024) for the reconciliation with Kaplan et al.
- E. Michaud, E. Liu, U. Girit, and M. Tegmark, *The Quantization Model of Neural Scaling* (2023); blog companion at `ericjmichaud.com/quanta`.
- A. Maloney, D. Roberts, and J. Sully, *A Solvable Model of Neural Scaling Laws* (2022); B. Bordelon, A. Atanasov, and C. Pehlevan, *A Dynamical Model of Neural Scaling Laws* (2024).
- The long lecture notes, *Physics of Deep Learning*, final section, with the survey table and worked solutions.